

Chat Application - Komplette Dokumentation

Inhaltsverzeichnis

1. Übersicht
 2. Systemanforderungen
 3. Installation
 4. Konfiguration
 5. Bedienungsanleitung
 6. API-Dokumentation
 7. Sicherheit
 8. Manuelle Tests
-

Übersicht

Die **Chat Application** ist eine webbasierte Echtzeit-Chat-Plattform, die mit Python Bottle Framework entwickelt wurde. Die Anwendung bietet eine moderne Benutzeroberfläche mit Session-Management und Echtzeit-Kommunikation.

Hauptfunktionen

- Moderne Benutzeroberfläche
 - Echtzeit-Chat mit automatischen Updates
 - Sicheres Session-Management
 - Automatischer Logout nach Inaktivität
 - Responsive Design
 - RESTful API
-

Systemanforderungen

Software-Voraussetzungen

- **Python 3.8+**
- **Bottle Framework**
- **Web-Browser** (Chrome, Firefox, Safari, Edge)
- **Internetverbindung** (für statische Ressourcen)

Abhängigkeiten

```
# requirements.txt  
bottle==0.13.4
```

Installation

1. Projektstruktur erstellen

```
SimpleChat/  
  WebApp.py  
  authentication  
    userbase.py  
  template/  
    index.html  
    chat_room.html  
    error404.html  
    error405.html  
  static/  
    style/  
      login.css  
      chat.css  
    js/  
      chat.js  
  test/  
    Programm_test.md  
    Programm_test.pdf
```

2. Abhängigkeiten installieren

```
pip install bottle
```

3. Anwendung starten

```
python main.py
```

4. Aufruf im Browser

```
http://localhost:8080
```

Konfiguration

Session-Einstellungen

```
# Standard-Session-Dauer: 5 Minuten  
SESSION_DURATION = 300 # Sekunden  
  
# Maximale Nachrichten im Speicher  
MAX_MESSAGES = 1000
```

Server-Einstellungen

```
# Standard-Port und Host
HOST = 'localhost'
PORT = 8080
DEBUG = True # Für Entwicklung
```

Bedienungsanleitung

1. Login

1. **Browser öffnen** und `http://localhost:8080` aufrufen
2. **Benutzername** und **Passwort** eingeben
3. Auf **“Login”** klicken

2. Chat verwenden

1. **Nachricht eingeben** im unteren Eingabefeld
2. **Enter** drücken oder auf **“Senden”** klicken
3. **Nachrichten werden automatisch** alle 2 Sekunden aktualisiert

3. Session-Management

- **Automatischer Logout** nach 5 Minuten Inaktivität
- **Warnung** erscheint 60 Sekunden vor Ablauf
- **Automatische Weiterleitung** zum Login bei Ablauf

4. Logout

- Auf **“Logout”** Button klicken
 - Oder Browser schließen
-

API-Dokumentation

Endpoints

POST /chat_room Beschreibung: User-Login und Session-Erstellung
Parameter: - `username` (string) - Benutzername - `password` (string) - Passwort
Response: - Erfolg: Chat-Room Template - Fehler: Redirect zu `/login?error=Invalid credentials`

GET /chat Beschreibung: Haupt-Chat-Seite
Session: Erforderlich
Response: Chat-Room Template oder Redirect zum Login

GET /api/messages **Beschreibung:** Holt Chat-Nachrichten

Parameter: - last_id (optional) - Letzte bekannte Nachrichten-ID

Response:

```
{
  "messages": [
    {
      "id": 1,
      "username": "max",
      "message": "Hallo!",
      "timestamp": "14:30:25"
    }
  ],
  "current_user": "max"
}
```

POST /api/send **Beschreibung:** Sendet neue Nachricht

Parameter: - message (string) - Nachrichtentext

Response:

```
{
  "success": true,
  "message_id": 42
}
```

GET /check_session **Beschreibung:** Überprüft Session-Status

Response:

```
{
  "valid": true,
  "username": "max",
  "remaining_time": 150
}
```

GET /logout **Beschreibung:** Beendet Session

Response: Redirect zu /login

Sicherheit

Session-Sicherheit

- **Session-Dauer:** 5 Minuten
- **Automatische Bereinigung** abgelaufener Sessions
- **HTTPOnly Cookies** verhindern XSS-Angriffe
- **Automatischer Logout** bei Inaktivität

Sicherheitsfeatures

1. **Session-Validation** bei jedem Request
2. **CSRF-Schutz** durch Session-Cookies
3. **Input-Validation** und HTML-Escaping
4. **Secure Cookie-Flags** (können für Production aktiviert werden)

Best Practices

- Passwörter nie im Klartext speichern
 - Session-Timeout nach Inaktivität
 - Sichere Cookie-Konfiguration
 - Input-Sanitization
-

Manuelle Tests

Testübersicht

Kategorie	Anzahl Tests	Status
Authentifizierung	3	Pendente
Chat-Funktionalität	4	Pendente
Session-Management	3	Pendente
API-Endpoints	3	Pendente
UI/UX Tests	3	Pendente
Sicherheitstests	3	Pendente
Gesamt	19	Pendente

Authentifizierungstests

Test 1: Erfolgreicher Login

Testfall-ID: AUTH-001

Priorität: Hoch

Dauer: 2 Minuten

Beschreibung: Valide Login-Daten verwenden

Voraussetzung: Server läuft auf localhost:8080

Testschritte: 1. Browser öffnen: **http://localhost:8080** 2. Benutzername eingeben: **max** 3. Passwort eingeben: **12345** 4. Auf "Login" klicken

Erwartetes Ergebnis: - [x] Weiterleitung zu /chat - [x] Chat-Oberfläche wird angezeigt - [x] Username wird oben links angezeigt - [x] Session-Cookie wird gesetzt

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 2: Fehlerhafter Login

Testfall-ID: AUTH-002

Priorität: Hoch

Dauer: 1 Minute

Beschreibung: Ungültige Login-Daten verwenden

Testschritte: 1. Browser öffnen: `http://localhost:8080` 2. Benutzername eingeben: `falscheruser` 3. Passwort eingeben: `falschespasswort` 4. Auf "Login" klicken

Erwartetes Ergebnis: - ☒ Bleibt auf Login-Seite - ☒ Fehlermeldung "Login Failed" wird angezeigt - ☒ Kein Session-Cookie gesetzt - ☒ Formular-Felder bleiben leer

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 3: Leere Login-Daten

Testfall-ID: AUTH-003

Priorität: Mittel

Dauer: 1 Minute

Beschreibung: Leere Formularfelder testen

Testschritte: 1. Browser öffnen: `http://localhost:8080` 2. Benutzername: (leer lassen) 3. Passwort: (leer lassen) 4. Auf "Login" klicken

Erwartetes Ergebnis: - ☒ Browser-Validation verhindert Absenden - ☒ Required-Fields werden markiert - ☒ Keine Server-Anfrage

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Chat-Funktionalität

Test 4: Nachricht senden

Testfall-ID: CHAT-001

Priorität: Hoch

Dauer: 2 Minuten

Voraussetzung: Erfolgreich mit `max` eingeloggt

Testschritte: 1. Im Chat-Fenster Text eingeben: "Hallo, das ist ein Test!" 2. Auf "Senden" klicken 3. Nachricht im Chat-Fenster beobachten

Erwartetes Ergebnis: - [x] Nachricht erscheint im Chat - [x] Username und Zeitstempel sichtbar - [x] Eingabefeld wird geleert - [x] Nachricht hat eindeutige ID

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 5: Lange Nachricht

Testfall-ID: CHAT-002

Priorität: Mittel

Dauer: 2 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Lange Nachricht eingeben (500+ Zeichen) 2. Auf "Senden" klicken 3. Formatierung überprüfen

Erwartetes Ergebnis: - [x] Nachricht wird korrekt gesendet - [x] Text wird umgebrochen - [x] Keine Layout-Probleme - [x] Max-Length wird eingehalten

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 6: Sonderzeichen in Nachricht

Testfall-ID: CHAT-003

Priorität: Hoch

Dauer: 2 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Nachricht eingeben: `<script>alert('xss')</script>` 2. Auf "Senden" klicken 3. Darstellung überprüfen

Erwartetes Ergebnis: - [x] Sonderzeichen werden escaped - [x] Keine Script-Ausführung - [x] Text wird sicher angezeigt - [x] Keine XSS-Verwundbarkeit

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 7: Leere Nachricht

Testfall-ID: CHAT-004

Priorität: Niedrig

Dauer: 1 Minute

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Leeres Eingabefeld lassen 2. Auf “Senden” klicken 3. Verhalten beobachten

Erwartetes Ergebnis: - [x] Keine Nachricht wird gesendet - [x] Keine Server-Anfrage - [x] Keine Fehlermeldung

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Session-Management

Test 8: Automatischer Session-Timeout

Testfall-ID: SESSION-001

Priorität: Hoch

Dauer: 6 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. 5 Minuten warten (Session-Zeit) 2. Versuchen, Nachricht zu senden 3. Browser-Verhalten beobachten

Erwartetes Ergebnis: - [x] Warning vor Ablauf (bei 60 Sekunden) - [x] Automatischer Logout nach 5 Minuten - [x] Redirect zu Login-Seite - [x] “Session expired” Meldung

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 9: Manueller Logout

Testfall-ID: SESSION-002

Priorität: Hoch

Dauer: 1 Minute

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Auf “Logout” Button klicken 2. Browser-Verhalten beobachten 3. Versuchen, zurück zu navigieren

Erwartetes Ergebnis: - [x] Redirect zu Login-Seite - [x] Session-Cookie wird gelöscht - [x] Kein Zugriff mehr auf /chat__room - [x] Beim Zurück-Navigieren: Redirect zum Login

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 10: Multi-Tab Session

Testfall-ID: SESSION-003

Priorität: Mittel

Dauer: 3 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Chat in Tab 1 öffnen 2. Neuen Tab öffnen und zu /chat navigieren 3. In Tab 1 ausloggen 4. Tab 2 aktualisieren

Erwartetes Ergebnis: - [x] Tab 2: Redirect zur Fehler Seite 405 - [x] Konsistente Session-Verwaltung

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

API-Endpoints

Test 11: API Messages ohne Session

Testfall-ID: API-001

Priorität: Hoch

Dauer: 1 Minute

Testschritte:

`curl "http://localhost:8080/api/messages"`

Erwartetes Ergebnis: - [x] JSON Response mit Error - [x] {"error": "Session expired", "redirect": "/login?error=Session expired"} - [x] Status Code 200

Status: - [x] BESTANDEN - [] FEHLGESCHLAGEN - [] BLOCKIERT

Test 12: API Send ohne Session

Testfall-ID: API-002

Priorität: Hoch

Dauer: 1 Minute

Testschritte:

`curl -X POST "http://localhost:8080/api/send" -d "message=Test"`

Erwartetes Ergebnis: - [x] JSON Response mit Error - [x] {"success": false, "error": "Session expired", "redirect": "/login?error=Session expired"} - [x] Nachricht wird nicht gespeichert

Tatsächliches Ergebnis:

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 13: Check Session Endpoint

Testfall-ID: API-003

Priorität: Mittel

Dauer: 2 Minuten

Voraussetzung: Eingeloggt

Testschritte: 1. Session Cookie aus Browser kopieren 2. Command ausführen:

```
curl -H "Cookie: session_id=DEINE_SESSION_ID" "http://localhost:8080/check_session"
```

Erwartetes Ergebnis: - ☒ JSON mit Session-Info - ☒ {"valid": true, "username": "testuser", "remaining_time": 297} - ☒ Korrekte Zeit-Berechnung

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

UI/UX Tests

Test 14: Responsive Design

Testfall-ID: UI-001

Priorität: Mittel

Dauer: 3 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Browser-Fenster verkleinern (Mobile-Größe) 2. Layout auf verschiedenen Größen testen 3. Auf Smartphone/Tablet testen

Erwartetes Ergebnis: - ☒ Layout bleibt nutzbar - ☒ Text lesbar - ☒ Buttons erreichbar - ☒ Keine horizontalen Scrollbars

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 15: Tastatur-Navigation

Testfall-ID: UI-002

Priorität: Niedrig

Dauer: 2 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Mit Tab durch Elemente navigieren 2. Enter zum Senden testen

Erwartetes Ergebnis: - ☒ Logische Tab-Reihenfolge - ☒ Enter sendet Nachricht - ☒ Fokus sichtbar

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Der Zeilenumbruch mit Shift+Enter ist aktuell nicht funktional.

Sicherheitstests

Test 16: Direkter Chat-Zugriff

Testfall-ID: SEC-001

Priorität: Hoch

Dauer: 1 Minute

Testschritte: 1. Ohne Login zu http://localhost:8080/chat_chat navigieren

Erwartetes Ergebnis: - ☒ GET von der Seite ERROR 405 - ☒ Kein Zugriff auf Chat

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 17: Cookie-Manipulation

Testfall-ID: SEC-002

Priorität: Hoch

Dauer: 2 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Session-Cookie im Browser editieren 2. ungültige Session-ID setzen 3. Seite aktualisieren

Erwartetes Ergebnis: - ☒ Redirect zu Login - ☒ Ungültige Session wird erkannt - ☒ Kein Zugriff mit manipuliertem Cookie

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT

Test 18: XSS-Prävention

Testfall-ID: SEC-003

Priorität: Hoch

Dauer: 2 Minuten

Voraussetzung: Erfolgreich eingeloggt

Testschritte: 1. Nachricht mit HTML/JS eingeben: `<img src=x onerror=alert(1)>` 2. Nachricht senden 3. Verhalten beobachten

Erwartetes Ergebnis: - ☒ HTML wird escaped angezeigt - ☒ Keine Script-Ausführung - ☒ Text wird sicher gerendert

Status: - ☒ BESTANDEN - ☐ FEHLGESCHLAGEN - ☐ BLOCKIERT
